

PathPilot: An AI-Powered Career Recommendation and Resume Intelligence System for Engineering Students

Dr. J. Manoranjini
Professor, Department of Computer
Science and Engineering
Rajalakshmi Engineering college
drmanoranjinicse@gmail.com

Pavithra N
Department of Computer Science
and Engineering
Rajalakshmi Engineering college
pavithranarayanan.own@gmail.com

Navitha M
Department of Computer Science
and Engineering
Rajalakshmi Engineering college
mnavitha99@gmail.com

Neha K
Department of Computer Science
and Engineering
Rajalakshmi Engineering college
Nehakarthis77@gmail.com

Nehaa Sesh S
Department of Computer Science
and Engineering
Rajalakshmi Engineering college
nehaaseshs@gmail.com

Abstract— Choosing the career path and making a good resume is something that all Undergraduate Computer Science and Engineering students have to deal with. The current resume checkers give advice and do not take into account a student's background, skills or goals. Career counselling services offer fixed and not advice. This paper presents PathPilot. A web-based tool that helps with career guidance and resume analysis. It uses chat-based career counselling and resume review to solve these problems and offer a personal service. The tool was built using a Flask backend, an SQLite database, a Jinja2/JavaScript frontend and the Gemini large language model. Unlike resume checkers PathPilot creates a context based on the student's profile and skills including skills and knowledge projects, certifications and career goals. This context is then combined with the resume text. Given to the large language model. The model gives back a JSON object that has a score ATS score, resume quality score, target role score, analysis of skills gaps, keyword coverage before and after suggestions to keep the facts and get the right format and recommendations for roles and companies based on the career knowledge graph. The tool clearly shows the difference between skills the student didn't mention and skills the student doesn't have. It also avoids making up any data, achievements or certifications. This paper explains why PathPilot was created its design, database structure resume processing, design and how it was built. It talks about the benefits compared to resume checkers and gives a general evaluation of the system. More studies, with users are needed for data.

Keywords—career guidance system, resume analysis, applicant tracking system, skill-gap analysis, generative AI, Flask, natural language processing, student profiling

I. INTRODUCTION

Every year many students who study Computer Science and Engineering go into the job market not knowing what to do. These students face challenges when trying to find the right areas where they can use their skills and reach their career goals. They also worry about whether their resumes will pass the tests companies use to select candidates. Career counseling at schools is too general. Doesn't help each student individually. At the time the programs that check resumes only look at the information people give and rate them based on certain rules. These rules might not match what the company is looking for in terms of education, goals and long-term plans.

Two problems are still not solved by the tools that're available. First the feedback on resumes is not personal. A tool that checks resumes cannot know that a person already has a skill unless the skill is written down as a keyword. It also can't tell the difference between a skill that is missing from the resume and a skill that the person never learned. Second career help and resume reviews are usually tools. This means students have to look at the advice from a chatbot about careers and the resume feedback separately and then try to make sense of both on their own.

PathPilot was created to solve both of these problems in one place. The system keeps a student profile that includes educational background, skills with levels of experience, projects, certifications and career interests. This profile acts as the base for any resume check or career question. The platform brings together four features that're usually separate: (i) managing a career profile that is specific to each user, including education skills with levels, projects and certifications; (ii) a chatbot that gives career help; (iii) a

resume checker that is made for the systems companies use and gives several scores; and (iv) checking for missing skills and giving advice to improve the resume without losing any important information.

The rest of the paper is arranged like this. Section II looks at research. Section III explains the system and how it was designed. Section IV describes how the system works overall. Section V talks about the database setup. Section VI explains how the resume-checking process works. Section VII covers how the AI models are used and how the questions are written. Section VIII gives details about how the system was built. Section IX shows the results and what was learned. Section X ends with a conclusion and ideas, for what could come.

II. RELATED WORK

Automated resume screening has been widely studied in the context of Applicant Tracking Systems (ATS). These systems usually depend on keyword matching, expression-based section detection or simple parsing to determine if a resume should move forward to human review [1]. While they are good at handling numbers of applications keyword-based ATS systems often penalize resumes that use different terms to describe real experience. They also offer no explanation for why a resume was rejected.

The rise of transformer-based language models has greatly improved how machines understand text. Models like BERT [2] showed that contextual language representations could outperform methods like bag-of-words or TF-IDF on many natural language tasks. The attention mechanism introduced in [3] powers modern large language models today including those used for open-ended reasoning and text generation. Recently multimodal generative models such as Google's Gemini family [4] and instruction-tuned models similar to GPT-3 [5] have made it possible to get structured outputs like JSON-formatted evaluations from a natural-language prompt. This is done without training a classifier for every specific task.

Career-recommendation research has traditionally used collaborative filtering or content-based matching between a candidate's stated skills and a job taxonomy. One used example is the U.S. Department of Labor's O*NET database [6] which lists knowledge, skills and abilities associated with job roles. These taxonomies are helpful as a reference. They are static and do not assess a student's resume or writing quality directly. Chatbot-based advising systems have also been tested to make career guidance more accessible. However, most of these systems. Lack a persistent record of the student or are limited to answering common questions rather than analyzing entire documents.

When it comes to the tasks that PathPilot handles there has been some work using classical machine learning techniques

on student or candidate data to generate suggestions. For example Rangnekar et al. [7] Built a career-counseling system that matches a student's declared traits with career paths using decision trees and support-vector-machine classifiers. This shows that structured student data can be used effectively to make recommendations without needing a language model. In the area of resume evaluation, Patel and Shah [8] introduced a semantic-similarity approach to improve tracking systems beyond basic keyword matching. Singh and Gupta [9] developed an AI system that scores resumes and suggests improvements.. None of these systems based on what is described in their publications use a consistent and verified profile of the student as PathPilot does. In fact PathPilot builds on the semantic similarity approach found in ATS scoring literature. It improves on it by using a user-specific profile as explained in Section V instead of relying only on the resume provided.

The PathPilot system has three advantages, over the earlier approaches. First it always uses the user-persistent profile to generate responses. Both the chatbot and the resume analyzer are not tools. They both draw on the information the user has entered in their profile. Second the system does not treat resume scoring, ATS-compliant optimization, skill gap analysis and career or company recommendations as tasks. It provides a unified analysis in one response. Finally the system applies factual-integrity constraints to stop the language model from making up skills or achievements that the user did not report. It also ensures that relevant keywords and scoring factors are not missed when identifying skill gaps.

III. PROPOSED SYSTEM

PathPilot is organized around a single guiding idea: every AI-generated recommendation, whether conversational or resume-based, should be conditioned on a structured, verifiable model of the student, not on the submitted text alone. The proposed pipeline, illustrated conceptually below, It all starts with the student profile and their current skill set. When a resume is provided, a number of artifacts are generated based on it. These artifacts include scores, the comparison of skills, the list of skill gaps, and recommendations. All these artifacts derive from the initial information provided in the form of a student's profile and resume evidence. This can be seen in the following conceptual structure: Student Profile → Current Skills → Resume → Target Career → AI Analysis → Resume/ATS Scores → Skill Comparison → Skill Gap → Learning Recommendations → Suitable Roles → Suitable Companies.

IV. SYSTEM ARCHITECTURE

PathPilot follows a layered, monolithic web-application architecture, chosen for its simplicity of deployment and ease

of maintenance in an academic project setting, while still cleanly separating concerns between presentation, application logic, business/service logic, and data. Fig. 1 illustrates the four principal layers of the system.

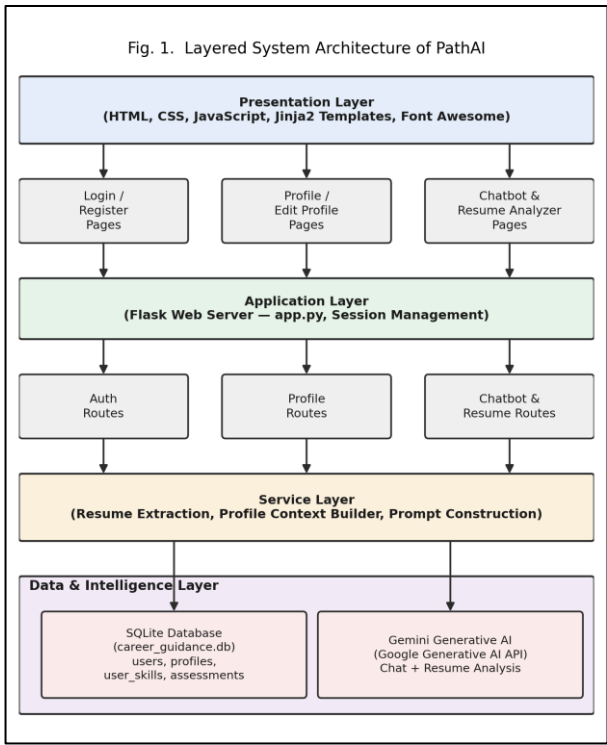


Fig. 1. Layered system architecture of PathPilot.

I see that the Presentation Layer is built with server-rendered Jinja2 templates. I see that the Presentation Layer uses a theme written in CSS. I see that the Presentation Layer adds JavaScript to support tasks, such as the chatbot interface and the resume-upload workflow. I see that the Presentation Layer also uses Font Awesome icons. I see that the Application Layer is a Flask web server. I see that the Application Layer keeps track of who's logged in by storing session['user_id'] and session['user_name']. I see that the Application Layer enforces authentication on protected routes. I see that the Application Layer routes each request to the handler for authentication, profile management, chatbot or resume analysis. I see that beneath the Application Layer the Service Layer holds logic that many routes reuse. I see that the Service Layer extracts text from a resume builds a student context and creates prompts for the Gemini API. I see that the Service Layer lets several parts of the system share the code. I see that the Data and Intelligence Layer contains a SQLite database called career_guidance.db. I see that the Data and Intelligence Layer stores every user, profile, skill and assessment record. I see that the Data and Intelligence

Layer also calls the Gemini generative-AI service for both the chatbot and resume analysis.

I see that this layering keeps AI-integration logic separate, from route handling. I see that because of that the language model, template or resume-parsing library can be swapped without editing Flask routes or templates. I see that the same service functions are reused by both the chatbot and resume analyzer.

V. DATABASE DESIGN

PathPilot is built as a monolithic web application. This approach was chosen because it is easy to deploy and easy to maintain. This makes PathPilot suitable, for a project. At the time PathPilot's architecture keeps the presentation, application logic, service functions and data components clearly separated.

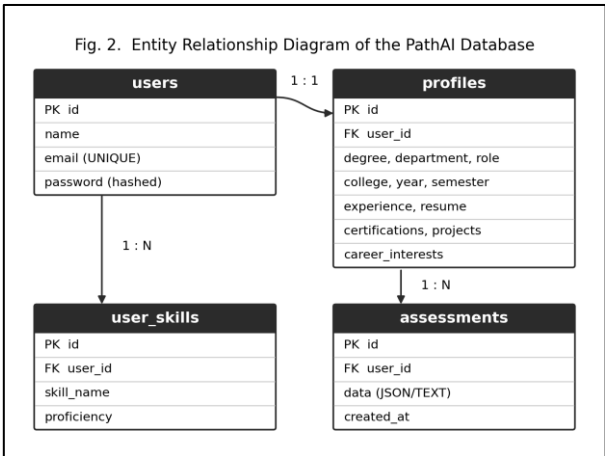


Fig. 2. Entity relationship diagram of the PathPilot database

Skills are kept in an one-to-many table instead of being stored in a single text field inside the profile. Each entry contains the skill name and the proficiency level, which can be Beginner, Intermediate or Advanced. This lets the application consider the actual level of expertise for a given skill than just checking if a keyword appears in a candidate's resume. As a result the accuracy of skill discovery during resume parsing improves.

The assessments table stores a JSON representation of each assessment performed on the user's data with the creation timestamp. Each record includes the skill_name and the proficiency level, such as Beginner, Intermediate or Advanced. This separation lets PathPilot differentiate between the candidate's level of expertise and mere keywords, which is essential for distinguishing declared skills from keywords during resume parsing.

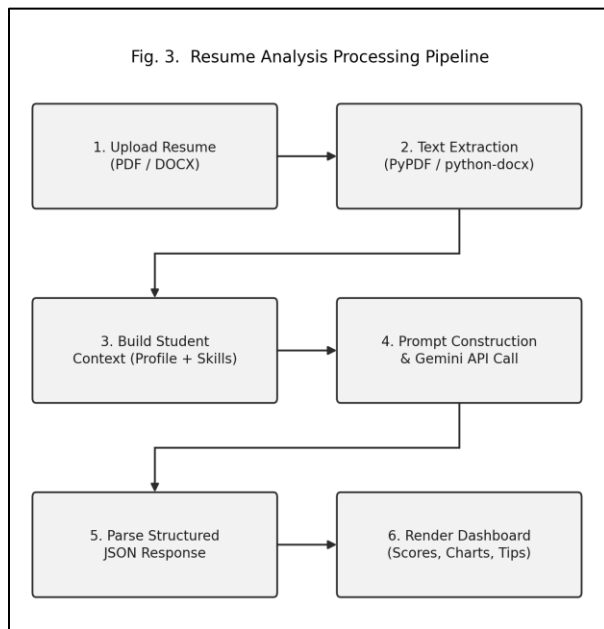
The assessments table holds a JSON snapshot of every analysis carried out for a user along with its creation

timestamp. This lets the user keep a history of their resume reviews for reference.

All foreign-key relationships link to user(id). All service-layer queries are filtered by the session identifier of the logged-in user so that data for students stays isolated

VI. RESUME ANALYSIS PIPELINE

The Resume Analyzer accepts resumes in PDF and DOCX formats through a POST request to the /resume/analyze endpoint. Users must be logged in to use this endpoint; otherwise they will get an HTTP 401 response, for access. The full resume processing pipeline is shown in Fig. 3.



Once the user uploads and checks their resume the system reads the text from the document. For a PDF the system reads each page one after the other. For a DOCX file it uses python-docx. If the system cannot read any text it will show an error. All the text that the system reads is joined with the user's verified profile. The profile lists qualifications, skills, projects, certifications, career interests, an optional target role and the job description. The system sends this data to the Gemini model. Waits for a JSON reply. The reply holds scores, strengths, problems, skill assessment, keywords, skill gaps, rewrite suggestions and recommendations. Finally the backend checks the results. Shows them on a clear dashboard.

VII. AI INTEGRATION AND PROMPT DESIGN

A. Grounding the Model in Verified Student Data

One design choice I want to highlight is to stop the model from guessing about a resume. Before asking any question or giving feedback the chatbot and resume analyzer give the

model a picture of the logged-in student's profile. This way the model looks first at the student's verified data in the database not at the resume text.

B. Explicit Factual-Integrity Constraints

The system prompt gives the model a set of instructions. Important the model must not create fake achievements, numbers, certificates or work history. It must not ask the student to guess their skills. Also if a skill is missing from the resume the model must tell whether the skill is just not mentioned or the student truly lacks it because each case needs a response. Likewise the model should not suggest tools that are only trendy. Those tools must match the student's goals and current skills. Instead the model should recommend skills that fit the job the student wants match the student's level fill skill gaps and avoid overusing keywords. These rules are not generic. They are custom for each student based on their needs and the current analysis. Also when giving resume tips the model may rewrite text a bit. The new version must keep all claims. Use different words. When recommending companies the model should look up the career knowledge base to avoid naming companies. The model's scores must be useful. Not promise a job. Important if the model finds conflicting data it must not fix it itself. It should point it out to the user for fixing.

C. Structured Output Contract

than letting the model format the answer in HTML or Markdown, the chatbot and resume analyzer ask Gemini to give structured data. For the chatbot this means a human-readable message that fits the chat interface. For the resume analyzer it means following the JSON schema. Thus the front end can show the data cleanly using badges, progress bars, charts and other tools of plain text from the model. This approach is very helpful for PathPilot giving an experience, than a simple chat.

VIII. IMPLEMENTATION

A. Authentication and Session Management

User registration and login are handled with securely hashed passwords. Once a user is authenticated, their identifier and display name are saved in the Flask session (session['user_id'], session['user_name']). Before accessing any secure routes, each request is checked against an active session to ensure that the correct user-specific information is retrieved. A new user must complete their profile before they can access the homepage whereas an existing user with a completed profile will be routed straight to the dashboard.

This overall process for the authentication and routing is summarized in Fig. 4.

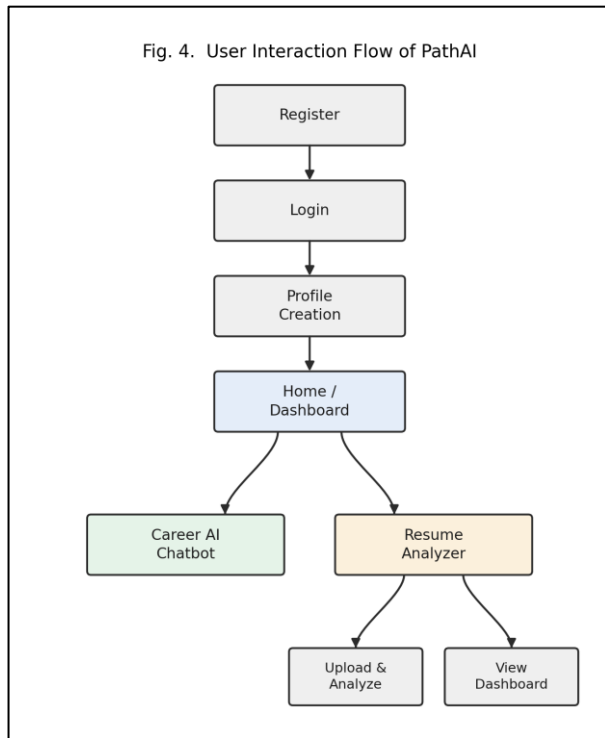


Fig. 4. User interaction flow of PathPilot.

B. Profiles

In the profiles section there are two templates that are used. These are called profile.html.. In the profile.html the student can see their information, education, skills, experience, projects, certifications, career interests and extracurricular activities. On the hand the edit_profile.html is only available when a student wants to make some changes to their profile. This is a way to make sure that the profile is not changed directly on the view page. When a student logs in the navigation bar has an icon with a dropdown menu. This dropdown menu shows the "View Profile" option of a logout option. This is done to stop students from leaving the page.

C. Conversational Career Chatbot

This conversational career chatbot can be used by the students to ask their questions in a way. The questions are then sent as a JSON payload through the POST /chat endpoint. A JSON response is also received. The chatbot is designed to answer questions about careers, jobs, required skills, relevant companies, ways to learn and tech- areas of interest. These areas include software development, data and automation testing. The chat interface has chat history, a typing indicator and suggestion chips. These chips help first-time users ask the questions.

D. Resume Analyzer Dashboard

The analyzer dashboard is the area where a student's resume is checked. Students first upload their resume in PDF format

for it to be analyzed. This dashboard has a resume preview, suggestions for improvement and a downloadable report. Students can reach the Resume Analyzer through a card on the landing page. This card opens up in a dashboard-style interface, not a simple upload form. They can upload their resume choose a target role if they want add a job description and start the analysis. When the returned JSON is shown the frontend displays the score, ATS score resume quality score and target-role match. These are shown using some form of indicators. The strengths and important issues are listed. The keyword analysis is shown with matched and missing keyword badges. The skill comparison is shown in a bar-style diagram. This diagram compares the resume-evidenced score to the expected one.

E. Career and Company Recommendation

Career and company suggestions are included in the JSON response, as the resume analysis. This means there is no recommendation system. The prompt that tells Gemini to create suggestions includes the student profile that was used to create the resume. It also includes the resume content if it was provided and the target role if it was specified. The JSON response that is generated has the recommended_skills, recommended_roles and recommended_companies fields. As explained in Section VII-B the recommended companies are taken from a career knowledge database that is carefully prepared. This means that they are not made up by the model but are taken from data. In the way the recommended roles are focused on the student's skill gaps and their interests. They are not just based on what's popular. This method makes sure that both the resume analysis and the career suggestions are based on the student profile data. This data is included in the prompt of the Gemini API request.

IX. RESULTS AND DISCUSSION

The modules, which we have been implementing (authentication, personal profile storage, skills storage with proficiency levels, career guidance chatbot, resume analytics backend with PDF/DOCX parsing), have been tested for their ability to store and process data specific to each user without interfering with other users' data, which is critical for this platform to maintain multi-user isolation. Table I below is an example of the skill comparison output for an example student profile parsed by the resume analyzer module. It demonstrates the level of proficiency indicated by the resume text for each skill, compared to the level required for a specific position. The most common source of discrepancy, prompting skill gap and learning recommendation reports in the chatbot is not the presence or absence of specific keywords in the resume, but rather the level of proficiency indicated in the resume text, if it differs from the required level, as demonstrated for the Statistics skill below.

TABLE I

Sample Skill Comparison Output

Skill	Resume Score	Expected Score
Python	80	90
SQL	70	85
Git	60	80
Statistics	30	80

Compared to traditional keyword-based ATS checkers, the profile-grounded approach used in PathPilot helps distinguish between two situations that may appear the same to a normal resume scanner: a skill the student has not learned and a skill the student possesses but has not mentioned in the resume. This distinction is important because the required action is different in each case: a missing skill requires a learning recommendation, while an existing skill not mentioned only requires a suggestion for improved resume wording. Combining profile-based analysis with clearly defined factual-integrity rules for the language model, PathPilot therefore offers resume improvement suggestions and skill recommendation based on the student's actually declared information instead of content generated or invented by the model.

The structured JSON response format also provides an additional advantage in that it allows for a single AI response to power multiple dashboard elements, such as the score indicators, keyword badges, skill-comparison charts, and prioritized improvement lists, removing the need for the frontend to interpret or reformat free-form AI-generated text and directly supporting the project's requirement that the student should not be presented with raw and unstructured AI output.

X. CONCLUSION AND FUTURE WORK

This paper introduces PathPilot. PathPilot is an AI-based platform that offers career consulting and resume feedback. PathPilot brings together student profiling, a career chatbot and resume scoring modules in a single Flask-based web application.

The main feature of PathPilot is grounded resume analysis. Unlike tools PathPilot does not try to score the resume on its own. PathPilot instead uses the students profile declared skills, proficiency and career interests as the reference. The PathPilot model is told not to invent achievements or company names. The PathPilot model is penalized if it ignores keywords. This approach keeps the focus on improvements. The PathPilot model outputs are in JSON format. This format lets PathPilot build a dashboard. The dashboard shows scores, skill gaps, keyword statistics and resume suggestions.

The main limitations of the study involve evaluation and the knowledge base. In Section IX the assessment of the PathPilot chatbot and resume analysis is mainly qualitative

and focuses on architecture. There was no controlled experiment to measure how accurate the scoring, skill extraction and recommendations are. A future research option is to run a user study. The study would compare the results of grounded resume analysis with labels from a comparable dataset. It could also evaluate how the score changes when using a skills profiler (see Section VII). Also although the PathPilot knowledge base covers professions it has not been updated with recent developments in the job market. This lack of updates threatens the relevance of the recommendations.

Future research directions suggested by the authors include updating the PathPilot knowledge base. They also suggest adding versions of a students resume. This would allow a resume-history view. Enable tracking of scoring improvements with each analysis by using the assessments table. Another idea is to use retrieval-augmented generation. This would help recommend companies and roles from an external knowledge base. The approach would also reduce hallucination risks. Finally a user study with engineering students could be conducted. The study would evaluate how effective PathPilot is compared to resume checkers that were common, at the time.

REFERENCES

- [1] H. Chen, Y. Zhang, and M. Li, "Artificial intelligence in recruitment: A survey of intelligent hiring systems," *IEEE Access*, vol. 11, pp. 54820–54839, 2023.
- [2] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [3] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [4] Google DeepMind, "Gemini: A family of highly capable multimodal models," Technical Report, Google, 2023.
- [5] T. Brown et al., "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.
- [6] U.S. Department of Labor, Employment and Training Administration, "O*NET OnLine occupational database," [Online]. Available: <https://www.onetonline.org>.
- [7] R. H. Rangnekar et al., "AI-based career counselling system using decision trees and SVM," *Int. J. Computer Applications*, vol. 182, no. 30, pp. 1–6, 2019.
- [8] S. Patel and K. Shah, "Semantic similarity-based ATS optimization for resume screening," *IEEE Access*, vol. 11, pp. 76540–76555, 2023.
- [9] R. Singh and V. Gupta, "AI-enabled resume assessment and recommendation system," *Journal of Information Technology Research*, vol. 16, no. 2, pp. 1–18, 2023.

- [10] Flask Development Team, “Flask documentation,” Pallets Projects, [Online]. Available: <https://flask.palletsprojects.com>.
- [11] SQLite Consortium, “SQLite documentation,” [Online]. Available: <https://www.sqlite.org/docs.html>.
- [12] Google, “Google Generative AI (Gemini API) documentation,” [Online]. Available: <https://ai.google.dev>.